

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_21

21. Strategy Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Strategy pattern. It is also known as the Policy pattern.

GoF Definition

It defines a family of algorithms, encapsulates each one, and makes them interchangeable. The Strategy pattern lets the algorithm vary independently from the clients that use it.

Concept

A client can select an algorithm from a set of algorithms dynamically at runtime. This pattern also provides a simple way to use the selected algorithm.

You know that an object can have states and behaviors. And some of these behaviors may vary among the objects of a class. This pattern focuses on the changing behaviors that can be associated with an object at a specific time.

In this example, you will see a `Vehicle` class. Once a `Vehicle` instance is created, you can add and set behaviors to this object. Inside the client code, you can replace the current behavior with a new behavior, too. Most interestingly, you will see that since the behaviors can be changed, the vehicle class does NOT define the behavior; it simply delegates the task to a particular object referenced by a vehicle. The complete implementation can make the concept clearer to you.

Real-Life Example

Consider a common scenario in a soccer match. Suppose team A has a 1–0 lead over team B toward the end of the game. In a situation like this, instead of attacking, team A becomes defensive in order to maintain the lead. At the same time, team B goes for an all-out attack to score the equalizer.

Computer World Example

Suppose you have a backup memory slot. If your primary memory is full but you need to store more data, you can use a backup memory slot. If you do not have this backup memory slot and you try to store the additional data into your primary memory, that data will be discarded (when the primary memory is full). In this case, you may receive an exception or you may encounter some peculiar behavior (based on the design of the program). So, a runtime check is necessary before you store the data. Here is another example for you:

- You can consider the interface `java.util.Comparator` in this context. You can implement this interface and provide different algorithms to do various comparisons using the `compare()` method. This comparison result can be further used in various sorting techniques. The `Comparator` interface plays the role of the strategy interface in this context.

Implementation

This demonstration focuses on different vehicle behaviors (or capabilities). You start with a simple class called `Vehicle`. To instantiate this class, you need to pass a `String` argument to indicate the type of vehicle. Here is the definition of the `Vehicle` class:

```
class Vehicle {
    String vehicleType;
    public Vehicle(String vehicleType) {
        this.vehicleType = vehicleType;
    }
    public String getVehicle() {
        return vehicleType;
    }
}
```

In this implementation, you focus on the changing behaviors of a vehicle only. This is why you see that once a vehicle object is created, it is associated with an `InitialBehavior` that simply tells you that in this state, the vehicle cannot do

anything special. But once you set a `FlyBehavior`, the vehicle can fly. When you set the `FloatBehavior`, it can float. Since a vehicle can show a particular behavior at a time and this behavior can vary, all these changing behaviors are maintained in a separate hierarchy as follows:

```
// Abstract Behavior
interface VehicleBehavior {
    void showDetail(Vehicle vehicle);
}

// Capability to float
class FloatBehavior implements VehicleBehavior {
    @Override
    public void showDetail(Vehicle vehicle) {
        System.out.println("The " + vehicle.getVehicle() +
            " can float now.\n");
    }
}

// Capability to fly
class FlyBehavior implements VehicleBehavior {
    @Override
    public void showDetail(Vehicle vehicle) {
        System.out.println("The " + vehicle.getVehicle() +
            " can fly now.\n");
    }
}

/**
 * This class used to show the initial
 * behavior of a vehicle. It cannot do
 * anything special.
 */
class InitialBehavior implements VehicleBehavior {
    @Override
    public void showDetail(Vehicle vehicle) {
        System.out.println("The " + vehicle.getVehicle() +
            " is in born state.");
        System.out.println("It cannot do anything special.\n");
    }
}
```

Now comes the most important class, which is often termed as a **context** class. `VehicleSupervisor` is the context class in this demonstration. This class adds/sets the capability of a vehicle.

```
class VehicleSupervisor {
    VehicleBehavior behavior;
    public VehicleSupervisor(VehicleBehavior behavior) {
        this.behavior = behavior;
    }
}
```

```

    }
    /**
     * You can use this method when you want to change
     * the "vehicle behavior" on the fly.
     */
    public void setVehicleBehavior(VehicleBehavior behavior){
        this.behavior = behavior;
    }
    /**
     * Delegates the behavior to the object referenced by
     * a vehicle.
     */
    public void displayDetail(Vehicle vehicle) {
        behavior.showDetail(vehicle);
    }
}

```

Inside the client code, you start with an initial behavior that is altered later using the `setVehicleBehavior(...)` method. The `displayDetail(...)` delegates the task to the particular object. This code demonstrate these activities. The important lines are shown in bold.

```

Vehicle vehicle = new Vehicle("airplane");
VehicleSupervisor supervisor = new VehicleSupervisor(new InitialBehavior
supervisor.displayDetail(vehicle);
System.out.println("Setting flying capability to it.");
supervisor.setVehicleBehavior(new FlyBehavior());
supervisor.displayDetail(vehicle);

```

Note You can use either a constructor or a setter method or both to set a vehicle behavior.

Class Diagram

Figure **21-1** shows the important parts of the class diagram.

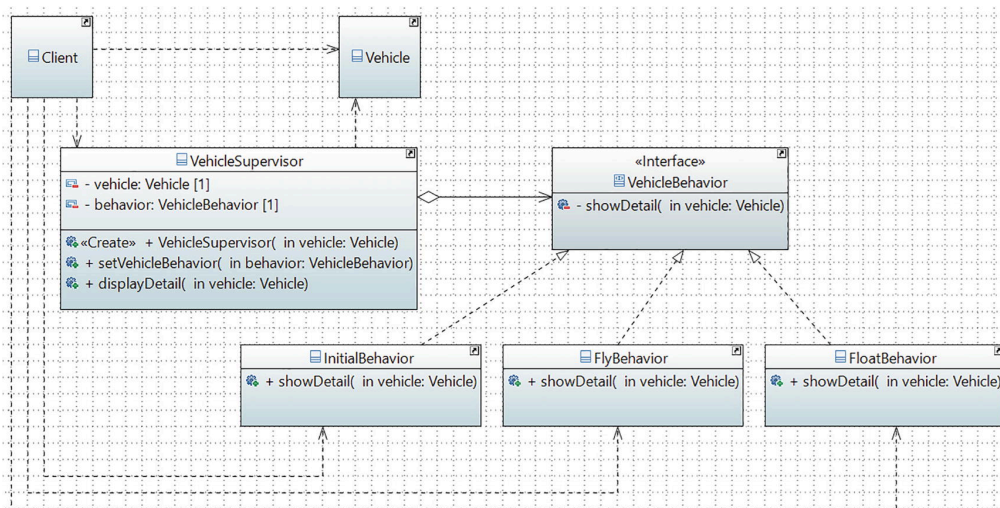


Figure 21-1 Class diagram

Package Explorer View

Figure **21-2** shows the high-level structure of the program.

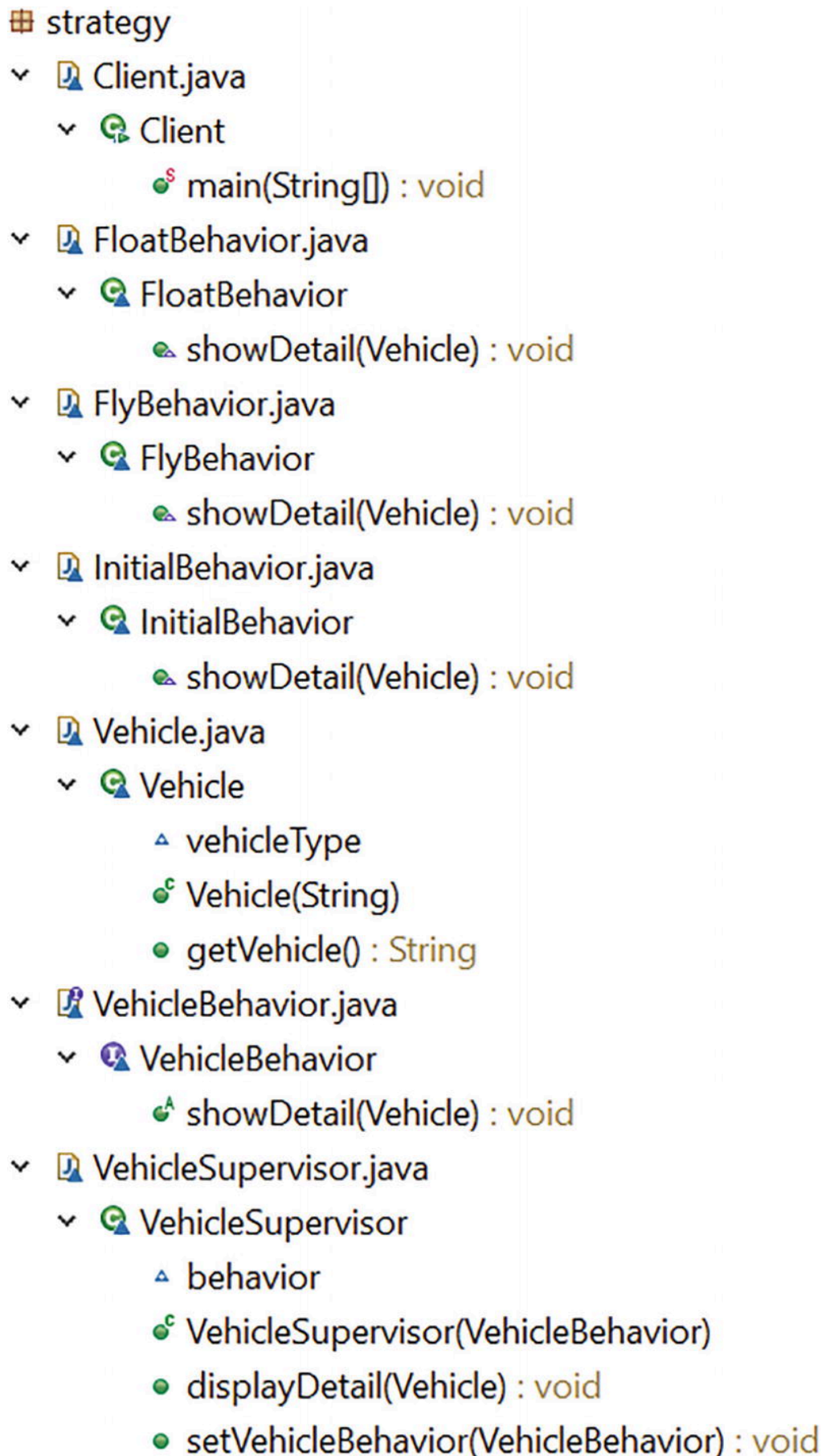


Figure 21-2 Package Explorer view

Demonstration

Here's the implementation. All parts of the program are stored inside the package `jdp3e.strategy`.

```
// VehicleBehavior.java
// Abstract Behavior
interface VehicleBehavior {
    void showDetail(Vehicle vehicle);
}

// FloatBehavior.java
// Capability to float
class FloatBehavior implements VehicleBehavior {
    @Override
    public void showDetail(Vehicle vehicle) {
        System.out.println("The " + vehicle.getVehicle() +
            " can float now.\n");
    }
}

// FlyBehavior.java
// Capability to fly
class FlyBehavior implements VehicleBehavior {
    @Override
    public void showDetail(Vehicle vehicle) {
        System.out.println("The " + vehicle.getVehicle() +
            " can fly now.\n");
    }
}

// InitialBehavior.java
/**
 * This class used to show the initial
 * behavior of a vehicle. It cannot do
 * anything special.
 */
class InitialBehavior implements VehicleBehavior {
    @Override

    public void showDetail(Vehicle vehicle) {
        System.out.println("The " + vehicle.getVehicle() +
            " is in born state.");
        System.out.println("It cannot do anything special.\n");
    }
}

// Vehicle.java
class Vehicle {
    String vehicleType;
    public Vehicle(String vehicleType) {
```

```

        this.vehicleType = vehicleType;
    }
    public String getVehicle() {
        return vehicleType;
    }
}
// VehicleSupervisor.java
/**
 * The VehicleSupervisor class represents the
 * context class in this example.
 */
class VehicleSupervisor {
    VehicleBehavior behavior;
    public VehicleSupervisor(VehicleBehavior behavior) {
        this.behavior = behavior;
    }
    /**
     * You can use this method when you want to change
     * the "vehicle behavior" on the fly.
     */
    public void setVehicleBehavior(VehicleBehavior behavior){
        this.behavior = behavior;
    }
    /**
     * Delegates the behavior to the object referenced by
     * a vehicle.
     */
    public void displayDetail(Vehicle vehicle) {
        behavior.showDetail(vehicle);
    }
}
// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Strategy Pattern Demo.***\n");
        Vehicle vehicle = new Vehicle("airplane");
        VehicleSupervisor supervisor=new
            VehicleSupervisor(new InitialBehavior());
        supervisor.displayDetail(vehicle);
        System.out.println("Setting flying capability to it.");
        supervisor.setVehicleBehavior(new FlyBehavior());
        supervisor.displayDetail(vehicle);
        System.out.println("Changing the vehicle behavior again.");
        supervisor.setVehicleBehavior(new FloatBehavior());
        supervisor.displayDetail(vehicle);
    }
}

```


Output

Here's the output:

```
***Strategy Pattern Demo.***
The airplane is in born state.
It cannot do anything special.
Setting flying capability to it.
The airplane can fly now.
Changing the vehicle behavior again.
The airplane can float now.
```

Q&A Session

21.1 It appears to me that you are complicating everything by focusing on changing behaviors. Also, I do not understand why you need the context class at all. You could simply use the inheritance mechanism and proceed. Can you please address these concerns?

If a particular behavior is common for all subtypes, it's okay to use inheritance. For example, you can make an abstract class and put the common behavior into it, so that all child classes get the common behavior. But the real power of the strategy comes into the picture when the behaviors can vary across the objects and maintaining them using inheritance is difficult.

For example, let's say, you start with different behaviors and you place them in an abstract class as follows:

```
public abstract class Vehicle
{
    public abstract void aboutMe();
    public abstract void floatBehavior();
    public abstract void flyBehavior();
    public virtual void defaultJob()
    {
        System.out.println("By default, I float.");
    }
}
```

Now let's say `Boat` and `Car` are two concrete classes that inherit from it. You understand that a `Boat` object should not fly, so inside the `Boat` class, you can simply override `flyBehavior` as follows:

```
public override void flyBehavior() throws Exception
{
    throw new Exception("Invalid vehicle feature.");
}
```

Similarly, a `Car` object should not float in water, so inside the `Car` class, you override the floating capability as follows:

```
public override void floatBehavior() throws Exception
{
    throw new Exception("Invalid vehicle feature.");
}
```

Now consider the case when you have lots of changing behaviors across objects like these. This kind of maintenance can be overhead for you.

Apart from this, let's consider a vehicle that has specialized features. If you simply put those special features in the abstract class, all other vehicle objects will inherit those features and need to implement them. But it is not over yet. Further, assume that there is a constraint on the `Boat` class that simply says that it cannot have any special behavior. Now you encounter a deadlock situation. If you implement this special method, you are violating the constraint, and if you do not implement it, the system architecture will be broken because as per the language construct, you need to implement the behavior. (Or you need to mark the class with the `abstract` keyword, but at the same time, remember that you cannot create an instance from an abstract class.)

To overcome this, you may say that you'll create a separate inheritance hierarchy where you'll make an interface to hold all the specialized features, and your classes can implement the interface if needed. This may solve the problem partially. This is because typically an interface contains abstract methods only (default methods in the Java interface were allowed from Java 8). So, there is no code reuse. Apart from this point, you may also note that an interface may contain multiple methods, but your class may need to implement only one of them. In a case like this, you need to consider the Interface Segregation Principle that you learned in Chapter 1. In the end, in either case, the overall maintenance becomes

tough. Apart from this, a specific behavior may also change in the near future and if so, you will need to track down all the classes that implement these behaviors.

In a situation like this, a context class acts as a savior for you. For example, the clients know that they should NOT set the fly behavior for a `Boat` class object. They also know that they should NOT set the set float behavior for a `Car` object. In short, they know which behavior is expected from the particular vehicle. If you want, you can guard the situation when a client by mistake sets an incorrect behavior to a vehicle, too.

To simplify this, the context class holds a reference variable for the changing behaviors of an object. At any particular moment, it can set the desired behavior and delegate the task to the appropriate behavior class.

So, a “has-a” relationship fits better than an “is-a” relationship in a similar situation. This is one of the primary reasons why most design patterns encourage you to prefer composition over inheritance.

21.2 What are the key advantages of using a Strategy design pattern?

Here are some key advantages:

- This design pattern makes your classes independent of algorithms. Here a class delegates the algorithms to the strategy object (that encapsulates the algorithm) dynamically at runtime. So, the choice of algorithms is not bound at compile time.
- It's easier to maintain your codebase.
- It's easily extendable.

You can refer to the answer to Q&A 21.1 in this context.

21.3 What are the key challenges associated with a Strategy design pattern?

The disadvantages can be summarized as follows:

- The addition of context classes causes more objects to exist in your application.
- Users of the application must be aware of different strategies; otherwise, the outputs may surprise them.

21.4 How does this pattern differ from the Template Method pattern?

You have asked a good question. These patterns have similarities. In the Strategy pattern, you can vary the entire algorithm using delegation. The Template Method pattern suggests you vary certain steps in an algorithm using inheritance but the overall flow of the algorithm is unchanged.

Previous chapter

[< 20. Memento Pattern](#)

Next chapter

[22. State Pattern >](#)